TripMind

AI Travel Planner with Real-Time Verification

System Architecture Document

Version 3.0

Three-tier architecture powering AI-generated travel itineraries
verified by 10 concurrent agents using free APIs and web scraping.

React 19 + Node.js Express + Python FastAPI + Claude AI

Last updated: March 2026

Contents

1 System Overview **2**
- 1.1 High-Level Architecture . . . 2
- 1.2 Technology Stack . . . 3

2 Frontend Architecture **3**
- 2.1 Component Tree . . . 3
- 2.2 9-Step Form Wizard . . . 3
- 2.3 State Management . . . 4

3 Trip Generation Pipeline **5**
- 3.1 Generation Flow . . . 5
- 3.2 Split-Prompt Strategy . . . 5
- 3.3 JSON Recovery . . . 6

4 Backend (Express API Gateway) **6**
- 4.1 Request Flow . . . 6
- 4.2 SSE Streaming Proxy . . . 6
- 4.3 Rate Limits . . . 7

5 Verification System **7**
- 5.1 Agent Orchestration . . . 7
- 5.2 Agent Details . . . 7
- 5.3 Fallback Chains . . . 8
- 5.4 Scraper Infrastructure . . . 8

6 Caching System **9**

7 Data Flow: End-to-End **10**

8 SSE Streaming Protocol **10**
- 8.1 Claude Streaming (Generation) . . . 10
- 8.2 Verification Streaming . . . 11

9 Results Display **11**

10 PDF Export **12**

11 Deployment Architecture **12**
- 11.1 Docker Containers . . . 12
- 11.2 Supported Platforms . . . 13

12 Security and Error Handling **13**

13 Parameter Flow **13**

14 Pydantic Data Models **14**

15 Configuration **14**

16 Cost Analysis **14**

1 System Overview

TripMind is an AI-powered travel planning application that generates personalized trip itineraries and then independently verifies every detail—hotel prices, flight costs, weather forecasts, safety advisories, place existence, reviews, and more—through 10 concurrent verification agents.

Key Design Principles

- **Split-prompt strategy:** Two Claude API calls (skeleton + days) to avoid token truncation on long trips.
- **SSE streaming:** Both trip generation and verification stream results to the browser in real time.
- **Fan-out/fan-in concurrency:** All 10 agents run in parallel via `asyncio.gather()`.
- **Graceful degradation:** The app works fully without the Python service (shows AI estimates).
- **Zero cost APIs:** All verification uses free APIs and web scraping—only the Claude API has a cost.
- **Fallback chains:** Scraping agents never fully fail; each has 2–3 fallback data sources.

1.1 High-Level Architecture

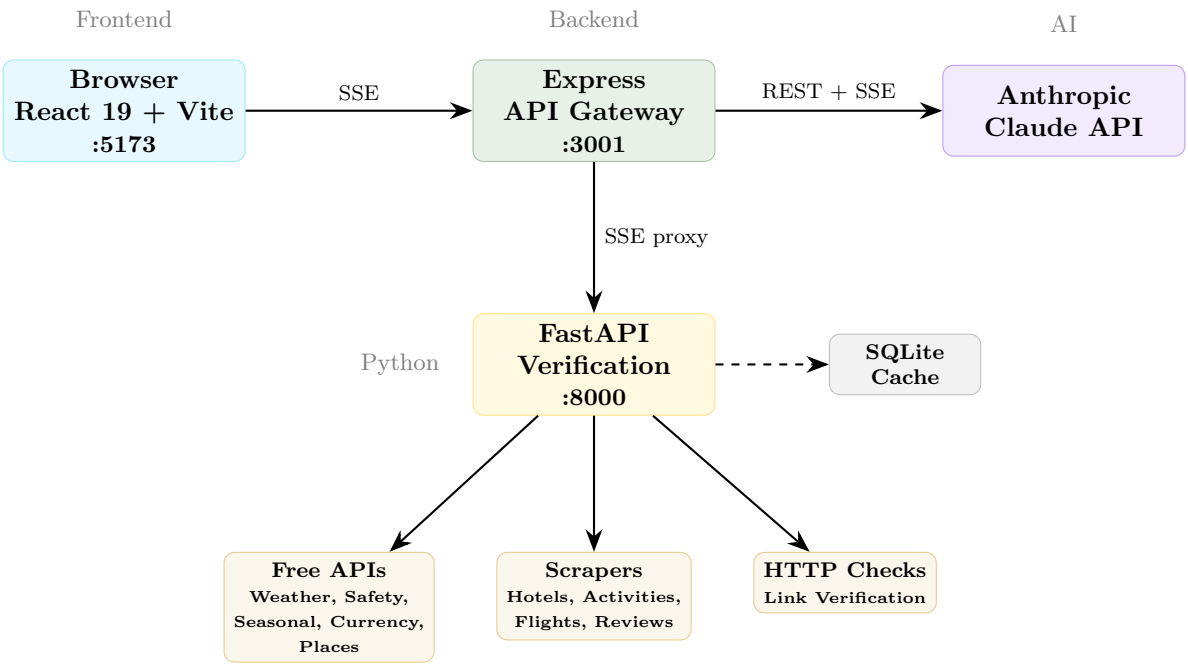

Figure 1: High-level system architecture showing the three services and their connections.

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	Component-based UI with HMR
Styling	CSS Modules	Scoped styles with design tokens
State	React Context	Form state, trip data, verification
Backend	Node.js + Express	API gateway, rate limiting, SSE proxy
AI	Anthropic Claude API	Trip itinerary generation (streaming)
Verification	Python FastAPI	Agent orchestration, SSE streaming
Scraping	Playwright + httpx	Browser automation and HTTP scraping
Caching	SQLite (aiosqlite)	TTL-based result caching
PDF	@react-pdf/renderer	Client-side PDF generation (lazy-loaded)

Table 1: Technology stack by layer.

2 Frontend Architecture

2.1 Component Tree

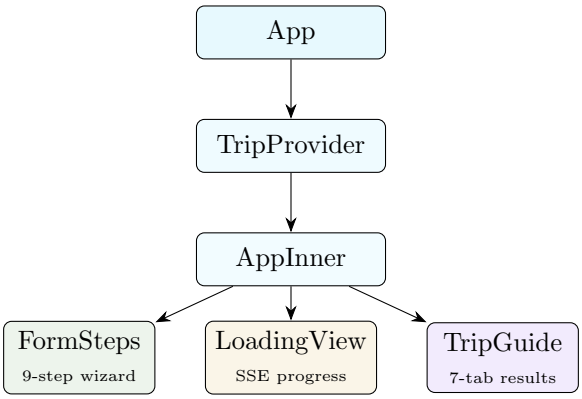

Figure 2: React component tree. Only one of FormSteps/LoadingView/TripGuide renders at a time.

The application has three mutually exclusive view states:

1. **FormSteps** — Shown when no trip is being generated and no results exist.
2. **LoadingView** — Shown during Claude API streaming (skeleton phase and days phase).
3. **TripGuide** — Shown when trip data is available, with verification running in the background.

2.2 9-Step Form Wizard

The form collects 24+ fields across 9 steps. Each step is a `switch` case in `FormSteps.jsx`:

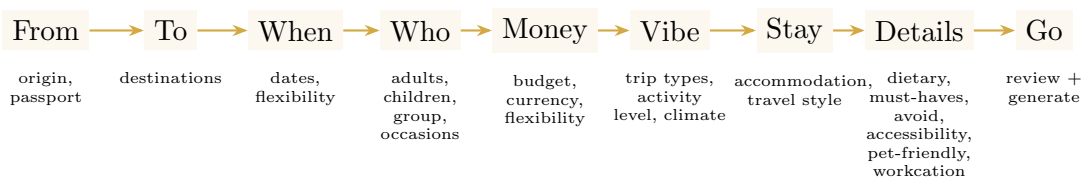

Figure 3: Form wizard steps and the fields each step collects.

Form Validation Rules

- Step 1 (From): Requires non-empty `origin`.
- Step 3 (When): Requires both `dateFrom` and `dateTo`.
- Step 5 (Money): Requires non-empty `budget`.
- All other steps: freely passable (no required fields).

2.3 State Management

All application state lives in a single React Context (`TripContext.jsx`):

State	Type	Purpose
`form`	Object (24+ fields)	All user inputs from the 9-step wizard
`step`	Integer (0–8)	Current wizard step
`loading`	Boolean	True during Claude API calls
`loadingPhase`	"skeleton" — "days"	Which generation phase is active
`streamText`	String	Accumulated streaming text (shown in LoadingView)
`tripData`	Object — null	Parsed trip plan (skeleton + days merged)
`verification`	Object — null	Verification results from all 10 agents
`verifying`	Boolean	True while agents are running
`error`	String — null	Error message for display

Table 2: Key state variables in TripContext.

3 Trip Generation Pipeline

Trip generation uses a two-phase "split-prompt" strategy to avoid token truncation on longer trips.

3.1 Generation Flow

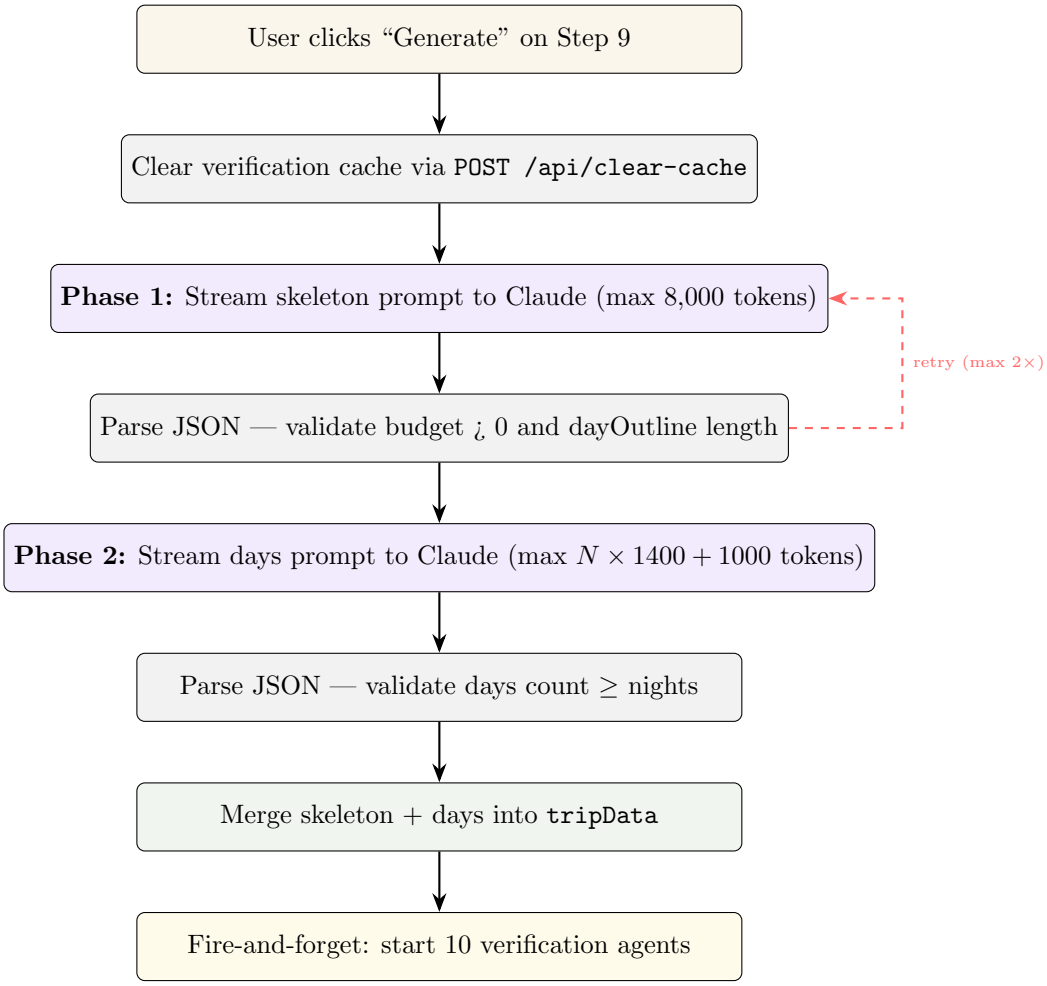

Figure 4: Two-phase trip generation pipeline with retry logic.

3.2 Split-Prompt Strategy

Why Two Calls?

A single Claude call for a 7-day trip with full activities, accommodations, flights, and budget would require 20,000+ tokens of output. At that length, the model often truncates the JSON mid-object, producing unparseable results. By splitting into skeleton (structure + budget + accommodations) and days (detailed activities), each call stays within a safe token budget.

Phase 1 — Skeleton Prompt generates:

- Trip title, tagline, hero image keyword
- Budget breakdown by category
- Flight options with search queries
- Accommodation options per city
- Day outline (day number, date, location, theme)

- Insider tips, packing highlights
- Instagram accounts, YouTube searches
- Suggested splurges with booking keywords

Phase 2 — Days Prompt receives the skeleton and generates:

- 3–4 activities per day with time, cost, Maps query, and booking keyword.
- One dining tip per day with Maps query.
- Image keywords for hero photos per day.

3.3 JSON Recovery

Claude occasionally produces malformed JSON (trailing commas, truncated output, unescaped characters). The `cleanAndParseJSON()` function applies a three-stage recovery:

1. Strip markdown code fences, find outermost {...}, remove trailing commas.
2. On parse failure: escape unescaped newlines inside JSON strings.
3. On second failure: repair truncated JSON—close open strings, remove partial key-value pairs, count unclosed braces/brackets with a state machine, and append closing characters.

4 Backend (Express API Gateway)

The Express server on port 3001 serves as a Backend-for-Frontend (BFF), proxying requests to Claude and the Python verification service.

4.1 Request Flow

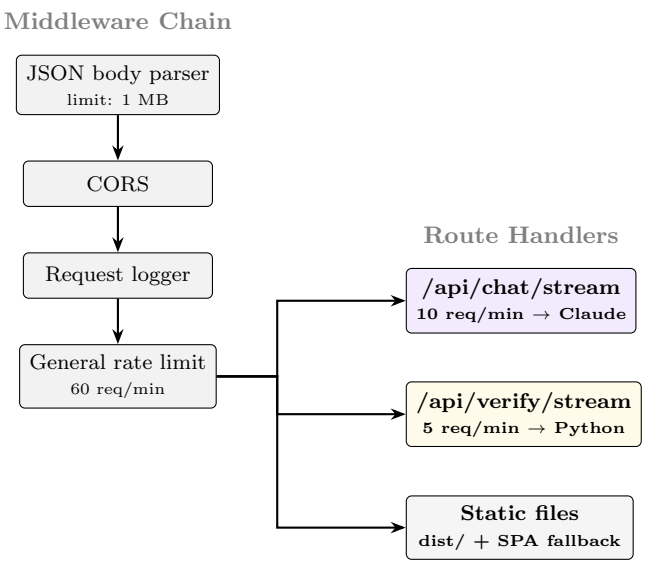

Figure 5: Express middleware chain and route handlers.

4.2 SSE Streaming Proxy

For Claude streaming (`/api/chat/stream`), the server:

1. Sets SSE headers: `Content-Type: text/event-stream`, `Cache-Control: no-cache`.
2. Calls the Anthropic API with `stream: true`.
3. Re-emits `content_block_delta` events as `data: {"text": "..."}`.
4. Sends `data: [DONE]` on `message_stop`.

For verification streaming (`/api/verify/stream`), the server performs a raw byte pipe—forwarding SSE chunks from the Python service directly to the browser without parsing (120-second timeout).

4.3 Rate Limits

Route	Window	Max Requests
All `/api/*` routes	60 seconds	60
`/api/chat`, `/api/chat/stream`	60 seconds	10
`/api/verify`, `/api/verify/stream`	60 seconds	5

Table 3: Express rate limiting configuration.

5 Verification System

The verification system is a Python FastAPI microservice that runs 10 agents concurrently to cross-check every detail of the AI-generated trip plan against real-world data.

5.1 Agent Orchestration

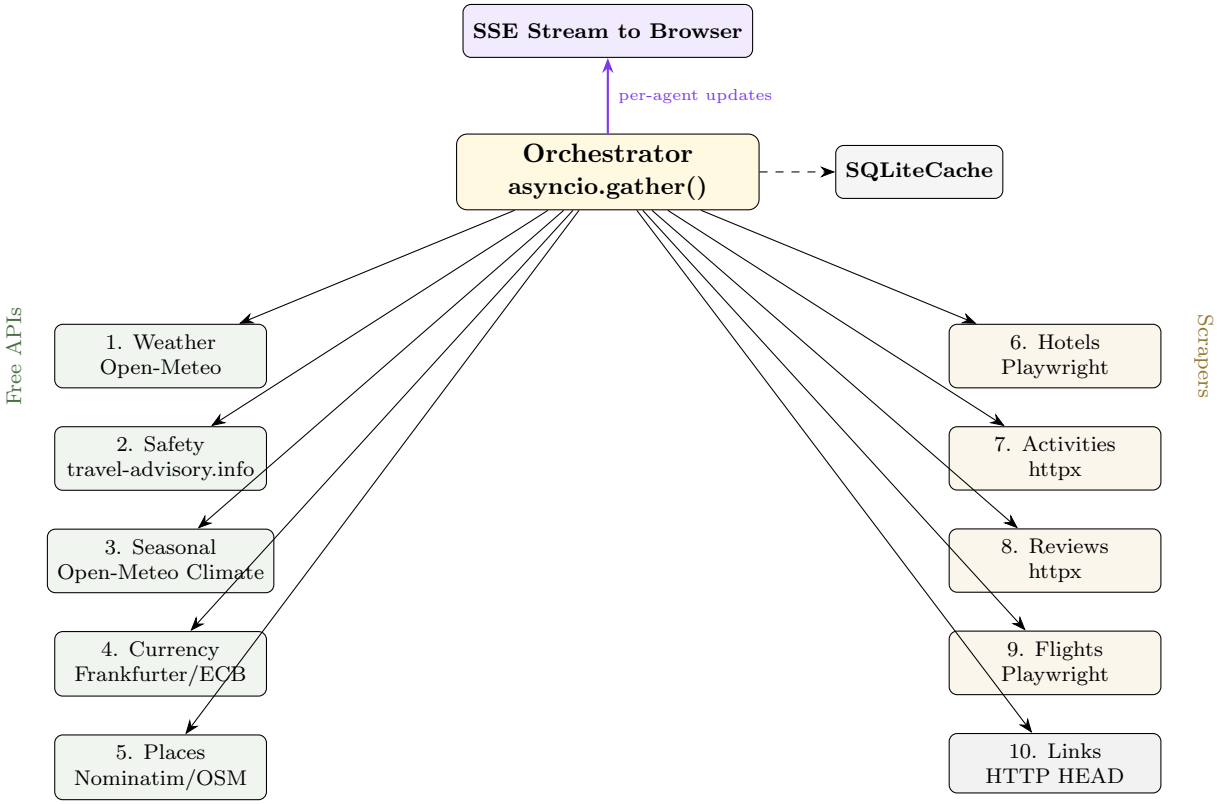

Figure 6: Fan-out/fan-in agent orchestration. All 10 agents run in parallel via `asyncio.gather()`.

5.2 Agent Details

Each agent follows a consistent pattern:

1. Check if the agent is enabled in `config.py`.

2. Build a cache key from the input parameters.
3. Check `SQLite cache` for a valid (non-expired) result.
4. On cache miss: call the underlying API or scraper.
5. On success: store the result in cache with the agent's TTL.
6. Return a Pydantic model (or `None` on failure).

Each agent coroutine is wrapped in `asyncio.wait_for(coro, timeout=45)` to prevent any single agent from blocking the pipeline.

#	Agent	Data Source	TTL	What It Verifies
1	Weather	Open-Meteo API	3h	Daily temp, rain, weather codes
2	Safety	travel-advisory.info	24h	Country risk score (0–5), advice
3	Seasonal	Open-Meteo Climate	7d	Monthly averages, tourism level
4	Currency	Frankfurter (ECB)	1h	Exchange rates vs. user's currency
5	Places	Nominatim / OSM	7d	Place existence, coordinates, address
6	Hotels	Google Hotels + Booking	6h	Real prices, ratings, booking URLs
7	Activities	Viator + GetYourGuide	12h	Activity prices, ratings, URLs
8	Reviews	Google Search	48h	Star ratings, review counts
9	Flights	Google Flights	1h	Prices, airlines, durations
10	Links	HTTP HEAD	24h	URL reachability

Table 4: All 10 verification agents with their data sources and cache TTLs.

5.3 Fallback Chains

Scraping agents are designed to never fully fail. Each has a chain of data sources:

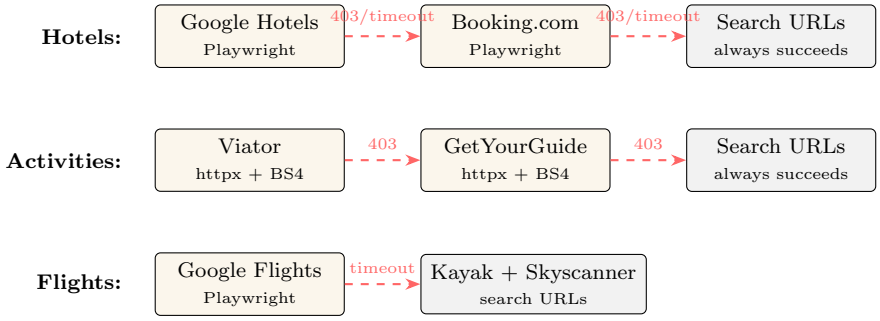

Figure 7: Fallback chains ensure scraping agents always return at least search URLs.

5.4 Scraper Infrastructure

The Playwright-based scrapers share a common infrastructure defined in `base_scraper.py`:

- **Shared browser instance:** A single Chromium browser is launched lazily and reused across all scraper calls, protected by an `asyncio.Lock`.
- **Per-request isolation:** Each scrape creates a new browser context (isolated cookies, storage) with a random user agent, then closes the context when done.
- **Anti-detection:** Injects JavaScript to hide `navigator.webdriver`, fake plugin count, set `window.chrome`, and override `navigator.languages`.
- **User-agent rotation:** Randomly selects from 6 user agents (Chrome, Firefox, Safari, Edge across Windows, macOS, and Linux).
- **Random delays:** 1–3 second random delay after each page navigation to mimic human behavior.

6 Caching System

All verification results are cached in a local SQLite database (`agents/cache.db`) to minimize redundant API calls and scraper runs.

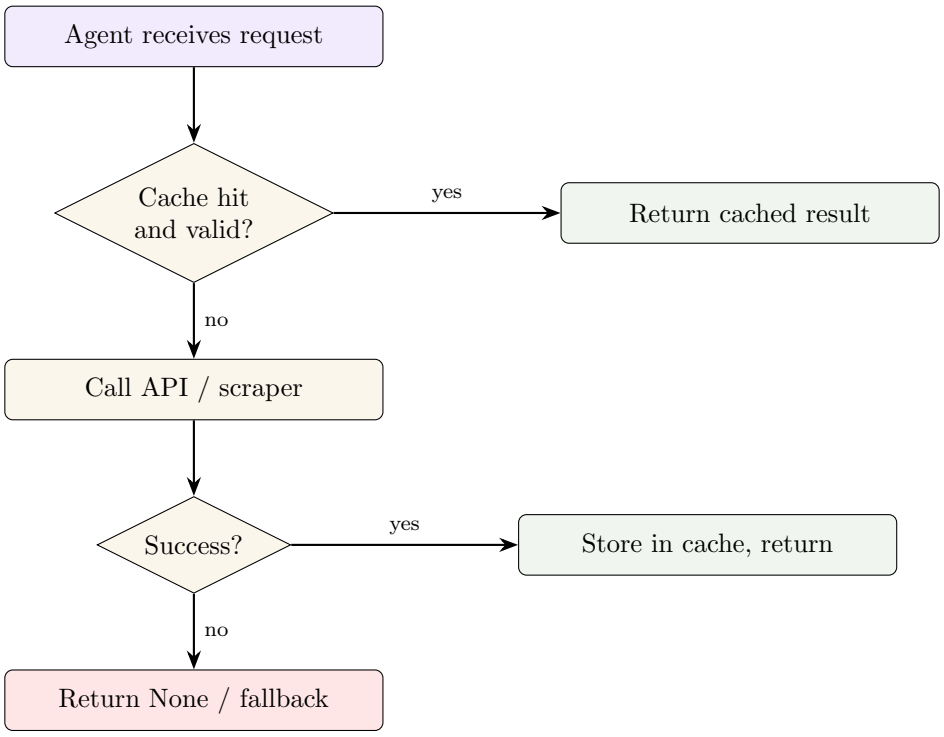

Figure 8: Cache lookup flow for each verification agent.

SQLite Schema

```sql
CREATE TABLE cache (
    key         TEXT PRIMARY KEY,
    value       TEXT NOT NULL,      -- JSON serialized
    created_at  REAL NOT NULL,      -- Unix timestamp
    ttl_hours   REAL NOT NULL
);
```

A row is valid when `time() < created_at + ttl_hours` $\times$ `3600`. Expired rows are purged on startup and lazily on read. The entire cache is cleared at the start of each new trip generation.

7 Data Flow: End-to-End

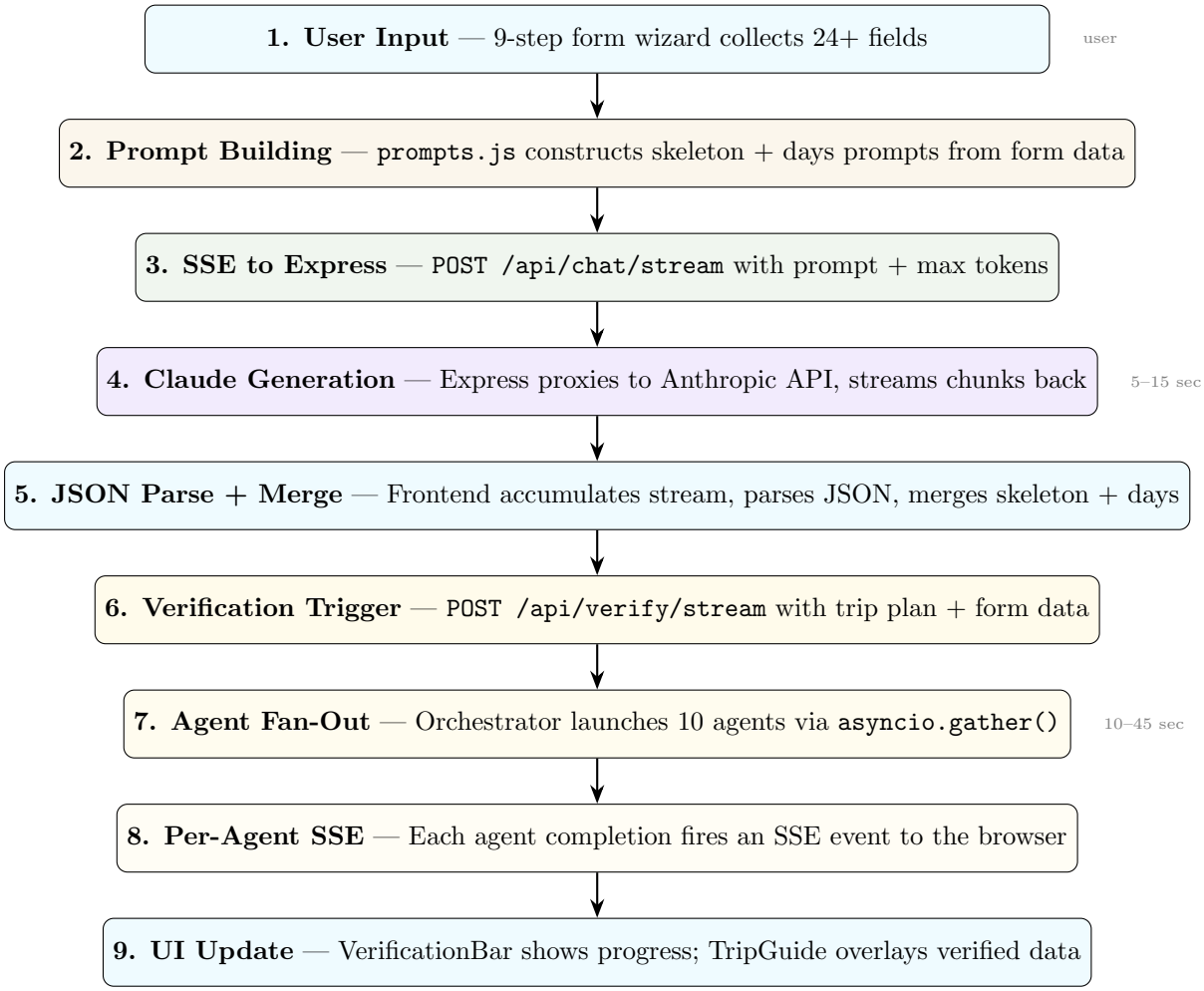

Figure 9: Complete end-to-end data flow from user input to verified trip display.

8 SSE Streaming Protocol

Both trip generation and verification use Server-Sent Events (SSE) for real-time streaming.

8.1 Claude Streaming (Generation)

Event	Format
Text chunk	`data: {"text": "partial JSON..."}`
Completion	`data: [DONE]`
Error	`event: error\ndata: {"error": "message"}`

Table 5: SSE event format for Claude streaming.

8.2 Verification Streaming

Event	Format
Agent result	`data: {"agent": "weather:Tokyo", "status": {...}, "snapshot": {...}}`
Completion	`data: [DONE]`

Table 6: SSE event format for verification streaming. Each agent completion fires one event.

The `status` object contains: `agent` (name), `status` (success/failed), `duration_ms`, `result_count`, and optionally `error`.

The `snapshot` is the cumulative `VerifyResponse` object with all results received so far.

9 Results Display

The `TripGuide` component renders the completed trip in a 7-tab layout:

Tab	Content
Map	Google Maps embed with route, numbered stops
Itinerary	Day-by-day activities with weather, ratings, verification badges
Stays	Verified hotel prices + AI accommodation options with booking links
Flights	Verified flight prices + AI estimates with search links
Budget	Currency conversion, breakdown table, live exchange rates
Suggestions	Curated splurges with Maps/booking links
Tips	Insider tips, packing list, Instagram accounts, YouTube searches

Table 7: The 7 tabs of the TripGuide results view.

Verification Overlay Strategy

Every piece of data in the UI displays a verification badge indicating its source:

- **Verified** (green) — Confirmed by a verification agent with real data.
- **AI Estimate** (purple) — Generated by Claude, not independently verified.

This transparency ensures users know which data points are independently confirmed.

10 PDF Export

PDF generation uses `@react-pdf/renderer` loaded via dynamic `import()` to keep the initial bundle small (the library is ∼1.5 MB).

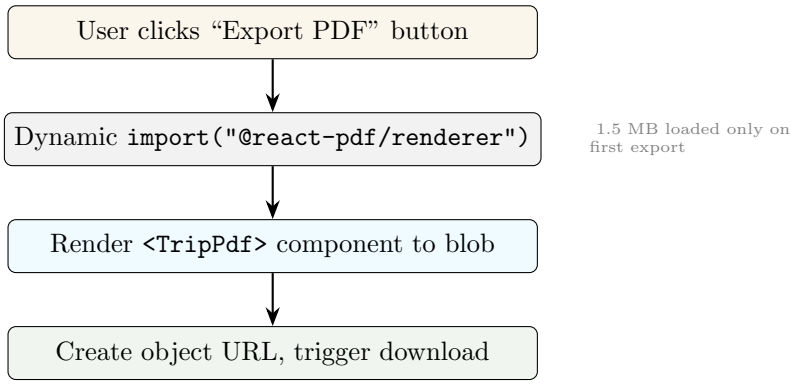

Figure 10: Lazy-loaded PDF export flow.

The PDF includes: trip header with stats, travel advisory, day-by-day itinerary, weather forecast, accommodations, budget breakdown, and travel tips.

11 Deployment Architecture

11.1 Docker Containers

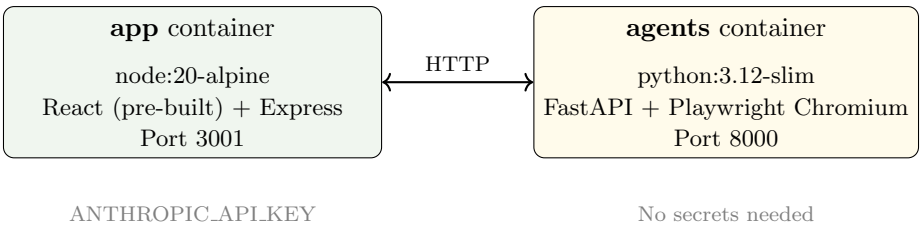

Figure 11: Docker Compose deployment with two containers.

The `app` container uses a multi-stage Dockerfile:

1. **Stage 1:** `node:20-alpine` builds the React frontend (`npm run build`).
2. **Stage 2:** `node:20-alpine` with production dependencies only, copies built `dist/` from Stage 1, runs `node server.js`.

The `agents` container installs system dependencies for Playwright, then installs Chromium and Python dependencies.

11.2 Supported Platforms

Platform	Configuration
Docker Compose	Two services, local development or self-hosted
Railway	Two services linked via internal networking
Render	Two Web Services (Node + Python)
Fly.io	Two apps with internal DNS (`.internal:8000`)
AWS ECS / Cloud Run / Azure	Container deployment from Docker Hub images

Table 8: Supported deployment platforms.

12 Security and Error Handling

- **API key validation:** The server validates the Anthropic key format on startup and rejects placeholder values.
- **Rate limiting:** Three tiers of rate limiting protect the Claude API and verification service.
- **Request size limit:** JSON body parsing capped at 1 MB.
- **Timeout guards:** 60-second timeout on non-streaming verify, 120-second on streaming, 45-second per-agent timeout.
- **CORS:** Open CORS for development; should be restricted in production.
- **No secrets in frontend:** The API key never leaves the server; the browser only talks to Express.
- **Graceful degradation:** If the Python service is down, the app works fully with AI estimates.
- **Scraper resilience:** Anti-bot 403s are expected and handled via fallback chains.

13 Parameter Flow

User form inputs flow to two destinations: the Claude prompts (for AI personalization) and the verification agents (for data lookup).

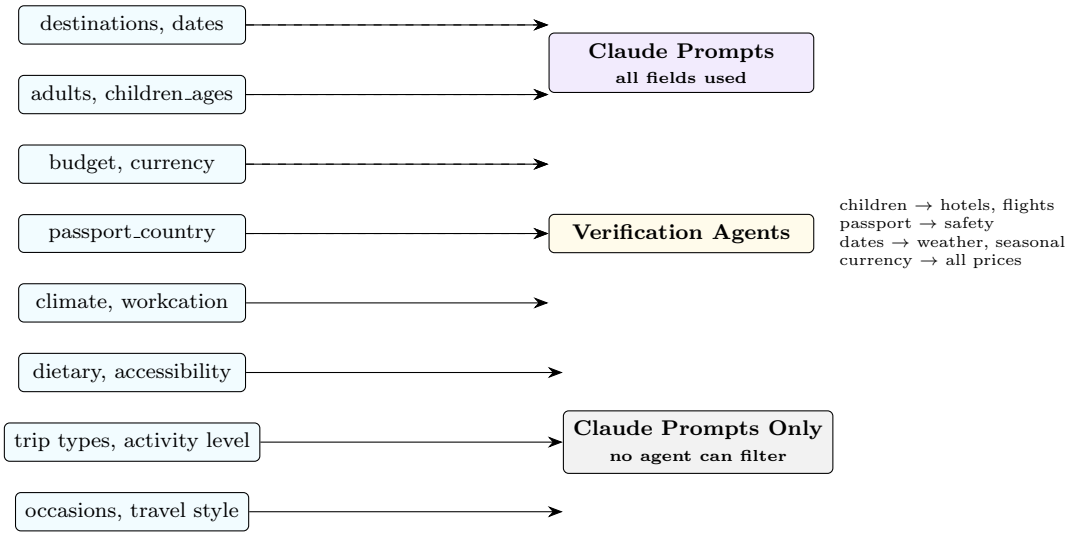

Figure 12: Parameter flow from form fields to Claude prompts and verification agents.

14 Pydantic Data Models

The Python verification service uses Pydantic models for all request/response schemas.

Model	Key Fields
`VerifyRequest`	`trip_plan`, destinations, dates, adults, children_ages, passport_country, currency, budget
`VerifyResponse`	`weather{}`, `safety{}`, `seasonal{}`, `currency`, `places[]`, `hotels{}`, `activities[]`, `reviews[]`, `flights[]`, `links[]`, `agent_statuses[]`, `verification_score`
`AgentStatus`	agent name, status (success/failed), duration_ms, result_count, error
`WeatherDay`	date, temp_high, temp_low, precipitation, weather_code, icon
`SafetyResult`	score (0–5), level, emoji, alerts[], advice
`HotelResult`	name, price_per_night, rating, review_count, source, booking_url
`FlightResult`	airline, price, duration, stops, source, booking_url
`PlaceVerification`	found (bool), lat, lon, address, confidence, maps_url
`ReviewResult`	place_name, rating (1–5), review_count, snippets[]

Table 9: Key Pydantic models and their fields.

15 Configuration

Agent behavior is controlled by `agents/config.py`:

- Each agent has an `enabled` flag that can be toggled off.
- Optional API keys (`SERPAPI_KEY`, `AMADEUS_KEY`, `VIATOR_API_KEY`, `GOOGLE_PLACES_KEY`) auto-upgrade agents from scraping to paid API providers when set.
- Scraping parameters: viewport size (1920×1080), delay range (1–3 seconds), 6 user agents for rotation.

16 Cost Analysis

Component	Cost	Notes
All 10 verification agents	\$0	Free APIs + web scraping
Claude API (per trip)	\$0.01–0.05	Two API calls: skeleton + days
Hosting (Docker)	Variable	Depends on platform

Table 10: Cost breakdown per trip generation.

Appendix: File Map

File	Responsibility
Frontend	
`src/main.jsx`	React entry point
`src/App.jsx`	App shell, view routing
`src/context/TripContext.jsx`	All state, generation logic
`src/prompts.js`	Claude prompt construction
`src/api.js`	HTTP client, SSE reader, JSON parser
`src/hooks/useVerification.js`	Verification SSE hook
`src/components/FormSteps.jsx`	9-step wizard
`src/components/TripGuide.jsx`	7-tab results display
`src/components/PdfExport.jsx`	Lazy PDF generation
Backend	
`server.js`	Express gateway, SSE proxy
Python Verification	
`agents/main.py`	FastAPI app, SSE endpoint
`agents/orchestrator.py`	Parallel agent dispatch
`agents/models.py`	Pydantic schemas
`agents/config.py`	Agent toggles, scraping config
`agents/cache.py`	SQLite TTL cache
`agents/agents/*.py`	10 agent wrappers
`agents/scrapers/*.py`	Playwright + httpx scrapers
`agents/free_apis/*.py`	Free API integrations

Table 11: Complete file map with responsibilities.